

Universidad de Antioquia

Facultad de Ingeniería

Informática II — Semestre 2026-1

Implementación de Tetris mediante Manipulación de Bits en C++

Integrantes:

Yoni Alejandro Serna Suarez

Brayan Stik Lopez Lopez

Profesor:

Augusto Salazar

marzo de 2026

1. Introducción

En este trabajo se desarrolló una versión simplificada del juego Tetris en C++, como parte del Desafío I de Informática II. El objetivo fue aplicar operadores a nivel de bits, memoria dinámica y programación modular dentro del entorno Qt.

El juego funciona por turnos: el sistema muestra el tablero, el usuario elige una acción y el tablero se actualiza. No hay animaciones en tiempo real, pero sí toda la lógica del Tetris clásico.

2. Análisis del Problema

El reto principal fue encontrar la forma de representar el tablero y las piezas usando bits en lugar de matrices convencionales. Esto era necesario porque el enunciado exige usar operaciones bitwise para los movimientos, colisiones y eliminación de filas.

Antes de programar fue importante entender bien las restricciones: el ancho del tablero debe ser múltiplo de 8, no se pueden usar `string` ni la STL, y todo debe trabajar con punteros y memoria dinámica.

2.1 Consideraciones para la solución

Las decisiones más importantes que tomamos antes de empezar a codificar fueron las siguientes. Primero, representar cada fila del tablero como un número entero sin signo, donde cada bit indica si una celda está ocupada o vacía. Elegir el tipo de dato según el ancho del tablero: definir las rotaciones de cada pieza como valores fijos en vez de calcularlas durante la ejecución, lo que simplifica el código considerablemente. Por último, separar el código en módulos distintos para cada funcionalidad: tablero, piezas, movimiento, colisiones, filas e interfaz.

3. Diseño de la Solución

El proyecto se organizó en varios módulos independientes. Cada uno se encarga de una parte del juego, lo que facilitó trabajar en cada funcionalidad por separado y detectar errores más fácilmente.

3.1 Tablero

El tablero se creó como un arreglo dinámico de punteros, donde cada posición apunta a una fila representada por un número entero sin signo. Cada bit de ese número representa el estado de una celda: en 1 si está ocupada, en 0 si está libre. Por ejemplo, en un tablero de 8 columnas el valor `0b00111100` corresponde a una fila donde las cuatro celdas del centro están ocupadas y las de los extremos están vacías.

3.2 Piezas

Las 7 piezas clásicas (I, O, T, S, Z, J, L) se representaron como conjuntos de filas de bits. Cada orientación posible de cada pieza se definió previamente como una constante, para no tener que calcular las rotaciones durante el juego. Esto hace el código más simple.

3.3 Organización del código

Se crearon archivos separados para cada parte del sistema. Esta separación permitió probar cada módulo de forma independiente antes de integrar todo, lo que ayudó a encontrar y corregir errores con mayor facilidad.

4. Algoritmos Implementados

4.1 Movimiento de piezas

El movimiento horizontal se implementa desplazando los bits de la pieza con los operadores « (izquierda) y » (derecha). Antes de aplicar el desplazamiento se verifica que el bit del borde correspondiente no esté activo, para evitar que la pieza salga del tablero. Este enfoque es eficiente porque no requiere recorrer arreglos, solo opera sobre el entero que representa la pieza.

4.2 Detección de colisiones

Para verificar si un movimiento es válido, se aplica la operación AND entre los bits de la pieza y los bits de la fila correspondiente del tablero. Si el resultado es distinto de cero hay superposición y el movimiento no se permite. El mismo principio se usa para detectar colisiones con el fondo del tablero.

4.3 Fijación de la pieza

Cuando la pieza ya no puede bajar, se fusiona con el tablero usando la operación OR. Esto activa los bits correspondientes en las filas donde cayó la pieza, dejando registro permanente de su posición.

4.4 Eliminación de filas

Una fila está completa cuando todos sus bits son 1. Para verificarlo se compara la fila con una máscara que tiene todos los bits activos según el ancho del tablero. Si coinciden, los punteros de las filas superiores se desplazan hacia abajo y la fila superior se reinicia en cero.

4.5 Game Over

El juego termina cuando al generar una nueva pieza en la parte superior del tablero se detecta una colisión inmediata, es decir, no hay espacio para colocarla desde el primer momento.

5. Problemas Encontrados

Durante el desarrollo surgieron varias dificultades. La más compleja fue la rotación de piezas: intentar calcularla matemáticamente resultó propenso a errores, así que se optó por definir todas las orientaciones de antemano y simplemente alternar entre ellas.

También hubo problemas con los bordes del tablero. Cuando la pieza llegaba al extremo y se intentaba desplazar, los bits “salían” del entero y generaban resultados incorrectos. Esto se resolvió verificando los bits extremos antes de permitir el movimiento.

Otro inconveniente fue con la memoria dinámica: al principio no se liberaban correctamente los arreglos con `delete[]`, generando fugas de memoria. Se corrigió asegurando que cada fila se libere individualmente antes de liberar el arreglo principal.

Por último, imprimir el tablero con la pieza activa superpuesta sin modificar el estado real requirió combinar los bits temporalmente solo para la visualización.

6. Evolución de la Solución

El proyecto no se desarrolló de una sola vez. Al inicio todo estaba en un solo archivo y la solución era más desordenada. Con el tiempo se fue separando en módulos y la estructura se volvió más clara.

Primero se trabajó la representación del tablero y la impresión en consola, para verificar que los bits funcionaban correctamente antes de agregar más lógica. Luego se agregaron las piezas y el movimiento, y finalmente se integraron las colisiones, la eliminación de filas y el Game Over.

Hacer pruebas en cada etapa antes de continuar fue clave para no acumular errores y detectar los problemas a tiempo.

7. Conclusiones

Este proyecto permitió aplicar de forma práctica conceptos importantes del curso. El uso de operadores a nivel de bits resultó ser una herramienta muy útil para representar el tablero y las piezas de manera eficiente, aunque al principio requirió un cambio en la forma de pensar el problema.

También quedó claro que dedicar tiempo al análisis antes de programar facilita mucho el proceso de codificación. Diseñar los módulos y pensar en la representación de los datos con anterioridad evitó muchos errores que de otro modo habrían sido difíciles de encontrar.